

Module 7: SELECT Statement

The **SELECT** statement is the most frequently used SQL command. It is used to retrieve data from one or more tables and forms the foundation of data analysis, reporting, dashboards, and business intelligence.



Introduction

Databases store data, but stored data has little value unless it can be retrieved and analyzed.

The **SELECT** statement allows us to:

- View records
- Retrieve specific columns
- Filter information
- Perform calculations
- Generate reports
- Prepare data for dashboards

Almost every SQL query starts with a SELECT statement.



Learning Objectives

After completing this module, you will be able to:

- ✓ Retrieve data from tables
 - ✓ Select specific columns
 - ✓ Use aliases
 - ✓ Remove duplicates
 - ✓ Retrieve limited records
 - ✓ Create calculated columns
 - ✓ Understand query execution basics
 - ✓ Write production-quality SELECT queries
-



Table of Contents

1. What is SELECT?

2. SELECT Syntax
3. SELECT *
4. Selecting Specific Columns
5. Column Aliases
6. DISTINCT
7. TOP Clause
8. Expressions and Calculations
9. Calculated Columns
10. Query Processing Order
11. Common Mistakes
12. Best Practices
13. Summary
14. Practice Questions

1 What is SELECT?

The **SELECT** statement retrieves data from database tables.

Example

Table:

EmployeeID	EmployeeName	Salary
101	Kunal	45000
102	Rahul	50000

Query:

```
SELECT *  
FROM Employees;
```

Output:

EmployeeID	EmployeeName	Salary
101	Kunal	45000
102	Rahul	50000

Why SELECT is Important

Used for:

- Reporting
- Dashboards
- Analytics
- Business Intelligence
- Data Extraction

2 SELECT Syntax

Basic syntax:

```
SELECT Column1, Column2
FROM TableName;
```

Example

```
SELECT EmployeeName,
       Salary
FROM Employees;
```

Output:

EmployeeName	Salary
Kunal	45000
Rahul	50000

SELECT Statement Structure

```
SELECT
FROM
WHERE
GROUP BY
HAVING
ORDER BY
```

These clauses will be studied in later modules.

3 SELECT *

The asterisk (*) means:

```
Retrieve all columns
```

Syntax

```
SELECT *  
FROM Employees;
```

Example

Table:

EmployeeID	EmployeeName	Salary	City
101	Kunal	45000	Delhi

Query:

```
SELECT *  
FROM Employees;
```

Output:

EmployeeID	EmployeeName	Salary	City
101	Kunal	45000	Delhi

Advantages

- ✓ Quick
- ✓ Useful during learning

Disadvantages

- ✗ Retrieves unnecessary data
- ✗ Slower for large tables

✗ Not recommended in production systems

4 Selecting Specific Columns

Instead of retrieving everything, select only required columns.

Example

```
SELECT EmployeeName,  
        Salary  
FROM Employees;
```

Output:

EmployeeName	Salary
Kunal	45000
Rahul	50000

Benefits

- ✓ Faster queries
 - ✓ Reduced memory usage
 - ✓ Better readability
-

Example

```
SELECT ProductName,  
        Price  
FROM Products;
```

5 Column Aliases

Aliases provide temporary names to columns.

Syntax

```
SELECT ColumnName AS AliasName  
FROM TableName;
```

Example

```
SELECT EmployeeName AS Name  
FROM Employees;
```

Output:

Name

Kunal

Rahul

Alias Without AS

```
SELECT EmployeeName Name  
FROM Employees;
```

Also valid.

Example

```
SELECT Salary AS MonthlySalary  
FROM Employees;
```

Multiple Aliases

```
SELECT EmployeeID AS ID,  
       EmployeeName AS Name,
```

```
Salary AS MonthlySalary
FROM Employees;
```

Why Use Aliases?

Improves readability.

Bad:

```
EmployeeName
```

Better:

```
Employee Name
```

Example:

```
SELECT EmployeeName AS [Employee Name]
FROM Employees;
```



DISTINCT

Used to remove duplicate values.

Example Table

City
Delhi
Delhi
Mumbai
Pune

Query:

```
SELECT DISTINCT City
FROM Employees;
```

Output:

City
Delhi
Mumbai
Pune

Without DISTINCT

```
SELECT City
FROM Employees;
```

Output:

Delhi
Delhi
Mumbai
Pune

Multiple Columns

```
SELECT DISTINCT City,
                 Department
FROM Employees;
```

Duplicates removed based on the combination.

Use Cases

- Unique customers

- Unique cities
- Unique products
- Reporting

7 TOP Clause

Returns a limited number of records.

Syntax

```
SELECT TOP 5 *  
FROM Employees;
```

Output:

First 5 rows.

Example

```
SELECT TOP 3  
    EmployeeName,  
    Salary  
FROM Employees;
```

TOP with Percentage

```
SELECT TOP 10 PERCENT *  
FROM Employees;
```

Returns:

```
Top 10% rows
```

TOP with ORDER BY

```
SELECT TOP 5 *  
FROM Employees  
ORDER BY Salary DESC;
```

Returns:

```
Highest 5 salaries
```

Real-World Use Cases

- Top Customers
- Top Products
- Top Employees
- Sample Data

Expressions and Calculations

SQL can perform calculations directly inside SELECT.

Example

```
SELECT 10 + 5;
```

Output:

```
15
```

Example

```
SELECT Salary + 5000  
FROM Employees;
```

Output

Salary + 5000

50000

55000

Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Example

```
SELECT 100 * 5;
```

Output:

500



Calculated Columns

Create new values dynamically.

Example

Table:

EmployeeName	Salary
Kunal	45000

Query:

```
SELECT EmployeeName,  
        Salary,  
        Salary * 12 AS AnnualSalary  
FROM Employees;
```

Output:

EmployeeName	Salary	AnnualSalary
Kunal	45000	540000

Example

```
SELECT ProductName,  
        Price,  
        Price * 0.18 AS GST  
FROM Products;
```

Business Applications

- Annual Salary
- Profit Calculation
- Tax Calculation
- Revenue Calculation

10 Query Processing Order

When SQL executes:

```
SELECT EmployeeName  
FROM Employees;
```

Database processes:

1. FROM
2. SELECT

Example:

```
SELECT EmployeeName  
FROM Employees  
WHERE Salary > 50000;
```

Actual execution:

1. FROM
2. WHERE
3. SELECT

Why Important?

Helps understand:

- Filtering
- Aggregation
- Query optimization

Common Mistakes

Misspelled Column Names

Bad:

```
SELECT EmployeName  
FROM Employees;
```

Error:

```
Invalid column name
```

Selecting Non-Existing Columns

Bad:

```
SELECT Bonus  
FROM Employees;
```

Error:

```
Column does not exist
```

Using SELECT *

Bad for production:

```
SELECT *  
FROM Employees;
```

Better:

```
SELECT EmployeeID,  
       EmployeeName  
FROM Employees;
```

Missing Commas

Bad:

```
SELECT EmployeeID  
EmployeeName  
FROM Employees;
```

Error.

Correct:

```
SELECT EmployeeID,  
        EmployeeName  
FROM Employees;
```

1 2 Best Practices

Select Only Needed Columns

Good:

```
SELECT EmployeeName,  
        Salary  
FROM Employees;
```

Avoid SELECT *

In production systems.

Use Meaningful Aliases

Good:

```
Salary AS MonthlySalary
```

Format Queries Properly

Readable SQL is easier to maintain.

Use Calculated Columns Carefully

Verify calculations.

Example of Well-Written Query

```
SELECT EmployeeID,  
       EmployeeName AS [Employee Name],  
       Salary,  
       Salary * 12 AS AnnualSalary  
FROM Employees;
```

Real-World Example

Employee Report:

```
SELECT EmployeeID,  
       EmployeeName AS [Employee Name],  
       Salary,  
       Salary * 12 AS AnnualSalary  
FROM Employees;
```

Output:

EmployeeID	Employee Name	Salary	AnnualSalary
101	Kunal	45000	540000
102	Rahul	50000	600000



Summary

In this module, you learned:

- ✓ SELECT Statement
- ✓ SELECT *
- ✓ Specific Columns
- ✓ Aliases
- ✓ DISTINCT

- ✓ TOP Clause
 - ✓ Expressions
 - ✓ Calculated Columns
 - ✓ Query Processing Basics
 - ✓ Best Practices
-

Practice Questions

Theory

1. What is the SELECT statement?
 2. What does SELECT * mean?
 3. Why should SELECT * be avoided in production?
 4. What is an alias?
 5. What is DISTINCT?
 6. What is the TOP clause?
 7. What are calculated columns?
 8. What is query processing order?
 9. What are arithmetic operators in SQL?
 10. Why select only required columns?
-

Practical Exercises

Task 1

Create:

```
Employees
```

Insert 10 records.

Retrieve:

```
SELECT *  
FROM Employees;
```

Task 2

Display only:

```
EmployeeName  
Salary
```

Task 3

Use aliases:

```
Employee Name  
Monthly Salary
```

Task 4

Find unique cities using:

```
DISTINCT
```

Task 5

Display:

```
Top 5 employees
```

using:

```
TOP
```

Task 6

Create calculated column:

```
AnnualSalary = Salary * 12
```

Challenge Project

Create:

Products

Table.

Generate a report showing:

- Product Name
- Price
- GST (18%)
- Final Price

using calculated columns.



Next Module

→ Module 8: Filtering Data with WHERE Clause

Topics Covered:

- WHERE Clause
- Comparison Operators
- Logical Operators
- BETWEEN
- IN
- LIKE
- Wildcards
- NULL Handling
- Advanced Filtering Techniques